Insurance Claims Frequency and Severity

McQueen Insurances

Contents

1 Executive Summary **1**

2 Data Preparation **1**

2.1 Data Transformation 2

2.2 Feature Engineering 16

3 Modelling **16**

3.1 Validation Framework 16

3.2 Frequency Modelling 16

3.3 Severity Modelling 16

4 Global Overview **16**

4.1 Importance of Features 16

4.2 Final Premium Structure 16

5 Bonus Analysis **16**

5.1 Alternative Tweedie Approach 16

5.2 Geospatial Insights 16

6 Conclusion **16**

1 Executive Summary

2 Data Preparation

```r
setwd(paste0(
    "C:/Users/MathieuDemarets/Documents/UCL Certificat/",
    "CodeRepositories/LACTU2150/project"
))
load("../data/mtpl_be.rda")
data <- mtpl_be
```

2.1 Data Transformation

2.1.1 Categorical Variables

```r
col_types <- sapply(data, class)

knitr::kable(
    data.frame(
        Type = col_types,
        Center = sapply(data, function(x) {
            if (is.numeric(x)) {
                mean(x, na.rm = TRUE) %>% round(4)
            } else {
                counts <- sort(table(x), decreasing = TRUE)
                names(counts)[1]
            }
        })
    ),
    caption = "Overview of variable types in the insurance claims dataset",
    booktabs = TRUE
) %>% kableExtra::kable_styling(full_width = TRUE, position = "left")
```

Table 1: Overview of variable types in the insurance claims dataset

	Type	Center
ID	integer	81616
NCLAIMS	integer	0.124
AMOUNT	numeric	200.8407
AVG	numeric	1625.589
EXP	numeric	0.8897
COVERAGE	character	TPL
FUEL	character	gasoline
USE	character	private
FLEET	character	N
SEX	character	male

AGEPH	integer	47
BM	integer	3.2687
AGEC	integer	7.375
POWER	integer	56.004
PC	integer	5270.7393
TOWN	character	CHARLERO
LONG	numeric	4.4072
LAT	numeric	50.7584

From this initial data loading, we can already notice that some columns have been asigned the wrong type:

- **COVERAGE, FUEL, USE, FLEET, SEX, and TOWN** should be set as unordered factor variables instead of character variables.
- **PC** should be converted from integer to an unordered factor variable.
- **BM** is the bonus-malus level and a higher score indicates a worse claim history, but the differences between levels are not necessarily uniform. We will thus treat it as an ordered factor variable.

We will also make sure that the reference levels are correctly set for the factor variables:

- Most frequent level for unordered factor variables
- Lowest level for ordinal factor variables

```
for (col in c("COVERAGE", "FUEL", "USE", "FLEET", "SEX", "TOWN", "PC")) {
    data[[col]] <- as.factor(data[[col]]) %>% fct_infreq()
}
data[["BM"]] <- factor(data[["BM"]], ordered = TRUE)
```

We can then assert that the categorical variables (ordered or unordered) have the correct reference levels.

```
assert_ref_level <- function(data, cols) {
    for (col in cols) {
        if (!is.factor(data[[col]])) stop(paste(col, "is not a factor variable"))
        if (is.ordered(data[[col]])) {
            if (as.numeric(levels(data[[col]]))[1] != min(as.numeric(as.character(data[[col]]))
            # Pay attention to turn labels as numeric and not the factor internal codes which
                stop(paste("Reference level for", col, "is not the lowest level"))
            }
        } else {
            if (
                levels(data[[col]])[1] !=
                    fct_count(data[[col]], sort = TRUE)$f[1]
            ) {
```

```
                stop(
                    paste(
                        "Reference level for", col,
                        "is not the most frequent level"
                    )
                )
            }
    }
    }
    cat("> Reference levels for", paste(cols, collapse = ", "), "are correctly set\n")
}
assert_ref_level(data,  c("COVERAGE", "FUEL", "USE", "FLEET", "SEX", "TOWN", "PC", "BM"))
```

> Reference levels for COVERAGE, FUEL, USE, FLEET, SEX, TOWN, PC, BM are correctly set

2.1.2 Numerical Variables Binning

Defining optimal binning for a numerical variable is a complex and important step. There are a set of naive approaches that can be used:

- Singleton bins: Each unique value of the numerical variable is treated as a separate bin.
- Equal-width bins: The range of the variable is divided into intervals of equal width.
- Equal-frequency bins: Quantiles of the variable are used to create bins with approximately the same number of observations.

For this project, we will try to implement a small algorithm based on weight of evidence (WoE) to automatically determine a suitable binning for numerical variables. The idea of WoE binning is to group values based on how strongly they are associated with the target variable (binary outcome encoded as 0 if claim = 0 and 1 if claim >= 1):

$$\text{WoE} = \ln \frac{\text{proportion of Good Outcomes in the bin}}{\text{proportion of Bad Outcomes in the bin}}$$

1. Use a quantile-based binning as baseline

The function `qbinning` is a simple algorithm that will start from the maximum number of bins specified by `max_bins` and iteratively reduce the number of bins until each bin contains at least one claim. This will ensure that the weight of evidence (WoE) can be calculated for each bin, as each bin will have at least one claim.

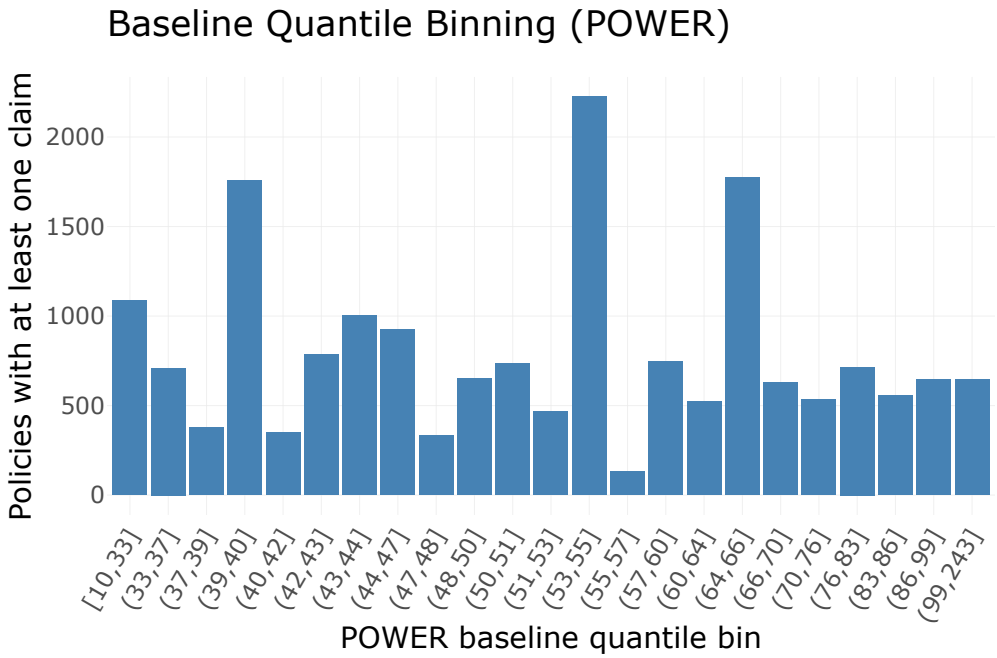
```
qbinning <- function(data, x, y, max_bins = 10) {
    "Quantile-based binning for numerical variable x with respect to binary target y"
    if (sum(data[[y]] >= 1) == 0) stop("No claims in the dataset")
    if (sum(data[[y]] == 0) == 0) stop("No non-claims in the dataset")
```

```
    if (!is.numeric(data[[x]])) stop("Variable x must be numeric")
    raw_data <- data
    temp_n <- max_bins
    temp_binning <- NULL
    baseline_bins <- NULL
    while (temp_n > 1) {
        temp_binning <- unique(
            quantile(raw_data[[x]], probs = seq(0, 1, length.out = temp_n + 1), na.rm = TRUE)
        )
        # check if there is at least one claim in each bin
        bin_claims <- tapply(
            raw_data[[y]],
            cut(raw_data[[x]], temp_binning, include.lowest = TRUE),
            function(z) sum(z >= 1))
        if (all(bin_claims > 0)) {
            baseline_bins <- temp_binning
            break
        }
        temp_n <- temp_n - 1
    }
    baseline_bins
}
```

```
bins_power <- qbinning(data, x = "POWER", y = "NCLAIMS", max_bins = 30)
counts <- tapply(
    data[["NCLAIMS"]],
    cut(data[["POWER"]], bins_power, include.lowest = TRUE),
    function(z) sum(z >= 1)
)

df_bins <- data.frame(
    bin = factor(names(counts), levels = names(counts)),
    count = as.numeric(counts)
)

p <- ggplot(df_bins, aes(x = bin, y = count)) +
ggtitle("Baseline Quantile Binning (POWER)") +
geom_col(fill = "steelblue") +
labs(x = "POWER baseline quantile bin", y = "Policies with at least one claim") +
theme_minimal() +
theme(axis.text.x = element_text(angle = 60, hjust = 1))
ggplotly(p)
```


Baseline Quantile Binning (POWER)
Policies with at least one claim
2000
1500
1000
500
0
[10,33]
(33,37]
(37,39]
(39,40]
(40,42]
(42,43]
(43,44]
(44,47]
(47,48]
(48,50]
(50,51]
(51,53]
(53,55]
(55,57]
(57,60]
(60,64]
(64,66]
(66,70]
(70,76]
(76,83]
(83,86]
(86,99]
(99,243]
POWER baseline quantile bin

2. Weight of Evidence (WoE) Calculation

Based on the formula above, the weight of evidence (WoE) for each bin can be calculated as the natural logarithm of:

- proportion of Good Outcomes in the bin $= \frac{\text{Number of Good Outcomes in the bin}}{\text{Total Number of Good Outcomes}}$
- proportion of Bad Outcomes in the bin $= \frac{\text{Number of Bad Outcomes in the bin}}{\text{Total Number of Bad Outcomes}}$

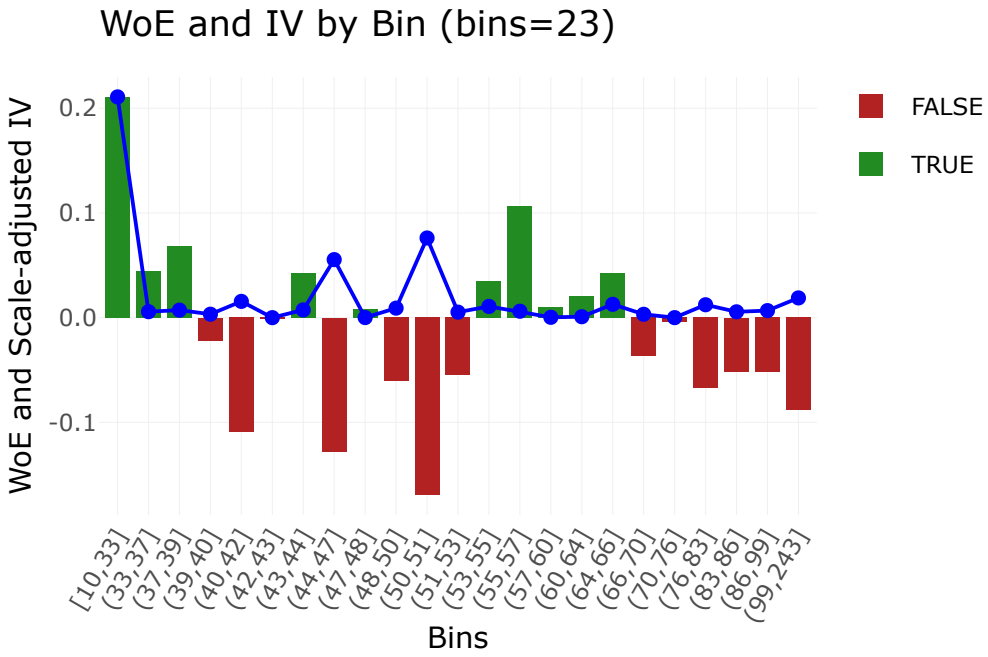
If the bin has as much good (no claim) as bad (at least one claim) outcomes, the WoE will be zero as there is very little predictive power in that bin. If the bin has more good outcomes than bad outcomes, the WoE will be positive, indicating that the bin is associated with a lower risk. Finally, if the bin has more bad outcomes than good outcomes, the WoE will be negative, indicating that the bin is associated with a higher risk.

```
woe_calc <- function(data, x, y, bins) {
    "Calculate Weight of Evidence (WoE) for each bin"
    woe_df <- data
    binned_woe <- tapply(
        woe_df[[y]],
        cut(woe_df[[x]], bins, include.lowest = TRUE),
        function(z) c(good = sum(z == 0), bad = sum(z >= 1))
    )
    binned_woe_df <- data.frame(
        bin = factor(names(binned_woe), levels = names(binned_woe)),
        good = sapply(binned_woe, function(x) x["good"]),
        bad = sapply(binned_woe, function(x) x["bad"])
    )
    rownames(binned_woe_df) <- NULL
    binned_woe_df$prop_good <- binned_woe_df$good / sum(binned_woe_df$good)
    binned_woe_df$prop_bad <- binned_woe_df$bad / sum(binned_woe_df$bad)
    binned_woe_df$woe <- log(binned_woe_df$prop_good / binned_woe_df$prop_bad)
    binned_woe_df$iv <- iv <- (binned_woe_df$prop_good - binned_woe_df$prop_bad) * binned_woe_
    return(binned_woe_df)
}

plot_woe_iv <- function(woe_df) {
    max_woe <- max(abs(woe_df$woe))
    max_iv <- max(abs(woe_df$iv))
    # Avoid dividing by zero when all values are zero
    scale_factor <- if (max_woe > 0 && max_iv > 0) {
        max_woe / max_iv
    } else {
        1
    }
    p <- ggplot(woe_df, aes(x = bin)) +
        geom_col(
            aes(y = woe, fill = woe > 0),
```

```
            width = 0.8,
            show.legend = FALSE
        ) +
        scale_fill_manual(
            values = c("FALSE" = "firebrick", "TRUE" = "forestgreen")
        ) +
        geom_line(
            aes(y = iv * scale_factor, group = 1),
            color = "blue"
        ) +
        geom_point(
            aes(y = iv * scale_factor),
            color = "blue",
        ) +
        scale_y_continuous(
            name = "WoE and Scale-adjusted IV"
        ) +
        labs(x = "Bins") +
        theme_minimal() +
        theme(
            axis.text.x = element_text(angle = 60, hjust = 1),
        ) +
        ggtitle(
            paste0("WoE and IV by Bin (bins=", length(woe_df$bin), ")")
        )
    ggplotly(p)
}
```

```
woe_ex <- woe_calc(data, x = "POWER", y = "NCLAIMS", bins = bins_power)
plot_woe_iv(woe_ex)
```


WoE and IV by Bin (bins=23)
WoE and Scale-adjusted IV
0.2
0.1
0.0
-0.1
FALSE
TRUE
[10,33]
(33,37]
(37,39]
(39,40]
(40,42]
(42,43]
(43,44]
(44,47]
(47,48]
(48,50]
(50,51]
(51,53]
(53,55]
(55,57]
(57,60]
(60,64]
(64,66]
(66,70]
(70,76]
(76,83]
(83,86]
(86,99]
(99,243]
Bins

3. Optimize based on the Information Value (IV)

The Information Value (IV) is a measure based on the Weight of Evidence (WoE) that quantifies the predictive power of a feature in distinguishing between good and bad outcomes. It is calculated as:

$$\text{IV} = \sum_{\text{bins}} (\text{proportion of Good Outcomes in the bin} - \text{proportion of Bad Outcomes in the bin}) \times \text{WoE of the bin}$$

The goal is to recursively merge the bin with lowest IV contribution with its neighboring bin, and evaluate the resulting IV. Then we can find the elbow point where further merging would lead to a significant loss of predictive power.

```r
iv_elbow <- function(data, x, y, bins) {
    "Recursively merge the bin with the lowest IV contribution with its neighboring bin with m

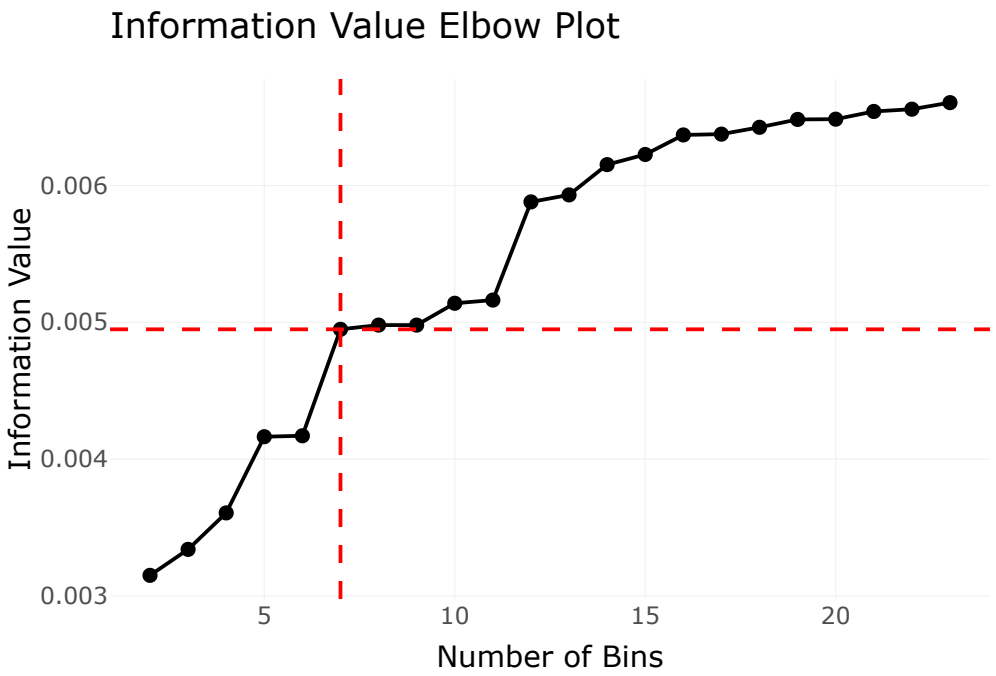
    temp_bins <- bins
    temp_n_bins <- length(bins) - 1
    temp_woe_df <- woe_calc(data, x, y, temp_bins)
    temp_iv <- temp_woe_df$iv
    temp_total_iv <- sum(temp_iv)

    iter_log <- data.frame(
        n_bins = temp_n_bins,
        iv = temp_total_iv,
        bins = paste(temp_bins, collapse = ","),
        diff = NA,
        diff_order = NA
    )

    while(temp_n_bins > 2) {
        bin_to_merge <- which.min(temp_iv)
        if (bin_to_merge == 1) {
            neighbor <- 2
        } else if (bin_to_merge == temp_n_bins) {
            neighbor <- temp_n_bins - 1
        } else {
            neighbor <- ifelse(temp_iv[bin_to_merge - 1] < temp_iv[bin_to_merge + 1], bin_to_m
        }
        to_merge <- sort(c(bin_to_merge, neighbor))
        temp_bins <- temp_bins[-to_merge[2]]
        temp_n_bins <- length(temp_bins) - 1
        temp_woe_df <- woe_calc(data, x, y, temp_bins)
        temp_iv <- temp_woe_df$iv
        temp_total_iv <- sum(temp_iv)
```

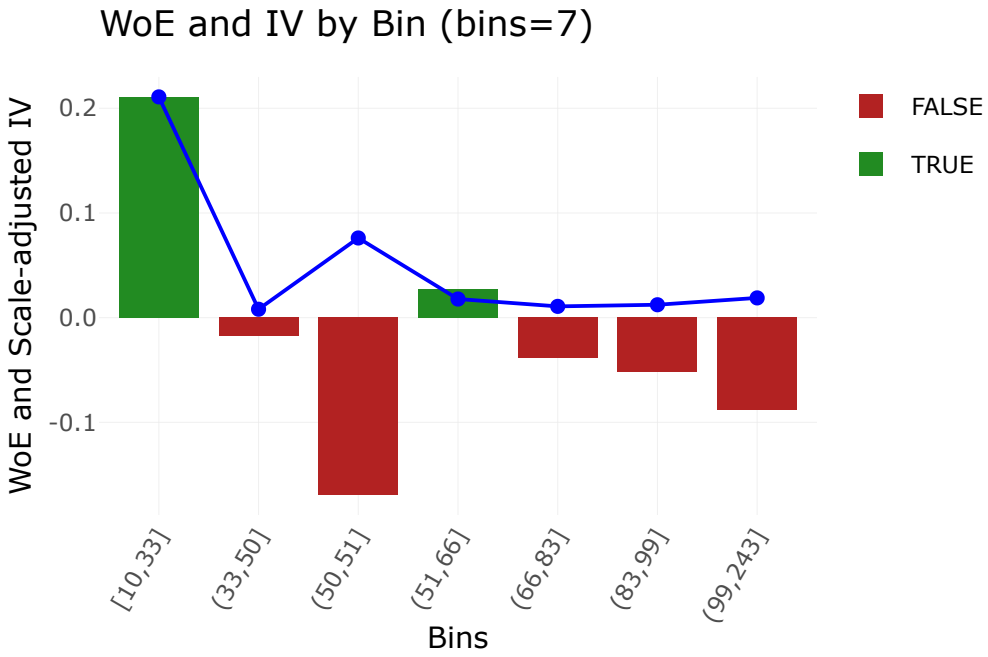
```
        iter_log <- rbind(iter_log, data.frame(
            n_bins = temp_n_bins,
            iv = temp_total_iv,
            bins = paste(temp_bins, collapse = ","),
            diff = NA,
            diff_order = NA
        ))
    }
    iter_log <- iter_log %>% arrange(n_bins)
    iter_log$diff <- c(NA, abs(diff(iter_log$iv)))
    iter_log$diff[is.na(iter_log$diff)] <- 0
    iter_log$diff_order <- rank(-iter_log$diff)
    return(iter_log)
}
plot_iv_elbow <- function(iv_elbow_result) {
    first_max_diff <- filter(iv_elbow_result, diff_order == 1)
    p <- ggplot(iv_elbow_result, aes(x = n_bins, y = iv)) +
        geom_line() +
        geom_point() +
        labs(title = "Information Value Elbow Plot", x = "Number of Bins", y = "Information Val
        theme_minimal() +
        geom_vline(xintercept = first_max_diff$n_bins, linetype = "dashed", color = "red") +
        geom_hline(yintercept = first_max_diff$iv, linetype = "dashed", color = "red")
    ggplotly(p)
}
```

```
result <- iv_elbow(data, "POWER", "NCLAIMS", bins_power)
plot_iv_elbow(result)
```


Information Value Elbow Plot
Information Value
0.006
0.005
0.004
0.003
5
10
15
20
Number of Bins

```r
rebuild_bins <- function(iv_elbow_result, n) {
    "Rebuild bins based on the IV elbow result for the specified number of bins"
    target_row <- iv_elbow_result %>% filter(n_bins == n)
    bins <- as.numeric(unlist(strsplit(target_row$bins, ",")))
    return(bins)
}
```

```r
opt_bins <- rebuild_bins(result, filter(result, diff_order == 1)$n_bins)
woe_ex <- woe_calc(data, x = "POWER", y = "NCLAIMS", bins = opt_bins)
plot_woe_iv(woe_ex)
```


WoE and IV by Bin (bins=7)
WoE and Scale-adjusted IV
0.2
0.1
0.0
-0.1
[10,33]
(33,50]
(50,51]
(51,66]
(66,83]
(83,99]
(99,243]
Bins
FALSE
TRUE

2.1.3 Statistical Approach

2.1.4 Graphical Approach

2.1.5 Machine Learning Approach

2.2 Feature Engineering

2.2.1 Geospatial Features

2.2.2 Polynomial Features

2.2.3 Interaction Features

3 Modelling

3.1 Validation Framework

3.1.1 Performance Metrics

3.1.2 Statistical Tests

3.1.3 Cross-Validation

3.2 Frequency Modelling

3.2.1 Naive and Full Models

3.2.2 GLM Models

3.2.3 GAM Models

3.2.4 Hurdle Models

3.2.5 Final Frequency Model

3.3 Severity Modelling

3.3.1 GLM Models

3.3.2 Final Severity Model

4 Global Overview

4.1 Importance of Features

4.2 Final Premium Structure

5 Bonus Analysis

5.1 Alternative Tweedie Approach 16

5.2 Geospatial Insights